

Lancer de rayon

1 Informations pratiques

Ce projet est à effectuer en binômes. Le code sera écrit en C++ et devra être compilable en ligne de commande ainsi :

```
g++ -g -Wall -Wextra -o prog *.cpp $(pkg-config --cflags --libs sdl2)
```

Pas de Makefile, de projet Eclipse, de projet Visual Studio, de projet Xcode ou quoi que ce soit d'autre.

Instructions

- Le code devra être indenté de manière uniforme.
- La définition des méthodes et des fonctions ne devront pas dépasser un nombre de lignes raisonnable (au plus 50 lignes).
- Le code devra être commenté en utilisant la syntaxe de Doxygen.
- Le rapport sera écrit en $\text{\LaTeX}$ et inclura les choix, les problèmes techniques qui se posent et les solutions trouvées, (la conception (dont un diagramme UML complet) et réalisation). Le soin apporté à la grammaire et à l'orthographe sera largement pris en compte.
- Le projet sera créé sous forme d'archive tar.xz ainsi :
 1. On vérifie que le code compile.
 2. On crée le répertoire *NOM1_NOM2_projet_rayon* où *NOM1* et *NOM2* sont les 2 noms de famille en majuscule. PAS de sous-répertoires.
 3. On y met **UNIQUEMENT** les fichiers source, fichiers d'en-tête, les fichiers .tex et .pdf du rapport et le fichier Doxyfile (qui crée la documentation)
 4. On compresse ce répertoire en ligne de commande ainsi :
 - *tar cvf NOM1_NOM2_projet_rayon.tar NOM1_NOM2_projet_rayon*
 - *xz -9 NOM1_NOM2_projet_rayon.tar*
 5. L'archive sera envoyée à vincent.torri@gmail.com avec pour sujet du mail *NOM1_NOM2_projet_rayon* avant le dimanche 4 janvier 2026 à 23h59. Un mail de confirmation sera envoyé dans les 24h. Si ce n'est pas le cas, relancez-moi.

IMPORTANT :

- Ne pas attendre le dernier moment pour envoyer l'archive.
- Si plusieurs binômes ont des codes trop similaires, leur note sera divisée par le nombre de binômes impliqués.
- Tout projet rendu en retard se verra attribuer la note de 0.
- Si le projet ne compile pas, la note sera 0. Je n'essaierai pas de corriger les erreurs.
- Non respect du sujet du mail : note diminuée de 1.
- Non respect du nom de l'archive : note diminuée de 1.
- Non respect du nom du sous-répertoire : note diminuée de 1.

Suggestions fortement recommandées :

- Créer un dépôt *Github* pour un travail collaboratif.
- Ajouter une intégration continue pour Linux et Windows (avec MinGW), pour s'assurer que le code compile sur ces deux plateformes.

2 Le tracé de rayon

L'obtention de la représentation d'une scène en 3D sur un écran peut s'obtenir de deux manières :

- La *rasterisation* : on crée un monde en 3D et on le projette sur l'écran à l'aide de formules de projection classiques. Cette technique est rapide (animation en temps réel possible) mais les effets réalistes tels les ombres doivent être simulés, d'où un rendu quelques fois peu réalistes de ce point de vue.
- Le *tracé de rayon* : on crée aussi un monde en 3D, mais au lieu de projeter le monde 3D sur l'écran, on suit le trajet d'un rayon partant de notre œuil, passant par un pixel de l'écran. Il suffit alors de suivre le rayon qui "rebondit" sur chacun des objets de la scène. Les images sont très réalistes, mais le calcul est plus coûteux et donc une animation en temps réel est difficile, malgré la puissance des ordinateurs actuels. Néanmoins, en disposant, soit de Windows 10 ou plus, soit un autre OS comme linux, les API Direct3D 12 et Vulkan (resp.), ainsi qu'avec une carte graphique adaptée, le lancé de rayon commence à devenir possible en temps réel.

On peut voir sur la Figure 1 (issue de Wikipedia) le principe du tracé de rayon.

L'algorithme général du tracé de rayon est le suivant :

1. Construire la scène avec les objets, la source de lumière et la caméra (*i.e.* l'œuil).
2. Calculer l'image.

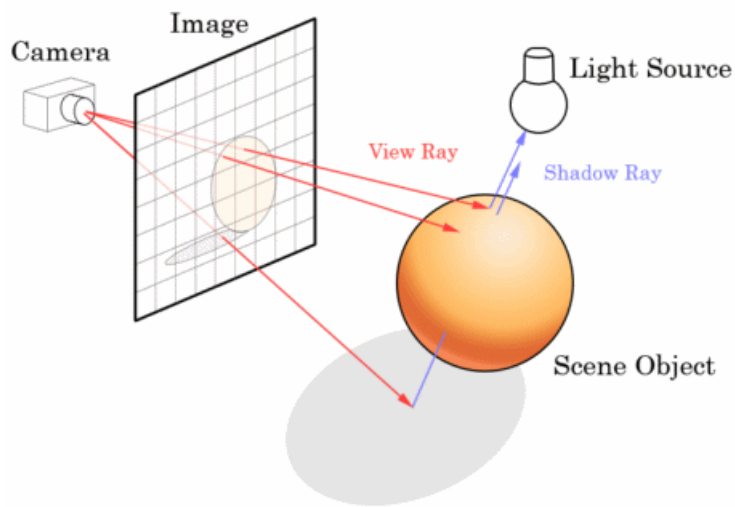


FIGURE 1 – Principe du lancé de rayon

3. Sauver l'image.

Un algorithme basique (qui devra être amélioré) pour le calcul de l'image peut être le suivant :

1. Déterminer la grille correspondant à l'image que l'on désire sauver
2. Pour chaque point de la grille :
 - (a) Déterminer l'équation de la ligne qui passe par l'œil et le point de la grille en question.
 - (b) Pour chaque objet de la scène :
 - i. Déterminer si la ligne intersecte l'objet.
 - ii. S'il y a intersection et si ce point est le plus proche de l'œil, on sauve ce point. L'intensité de la lumière sur le point d'intersection se fera grâce à la position de source de lumière.
 - iii. Si l'objet possède un indice de réflexion, on continue de suivre le rayon avec la ligne réfléchi.
 - (c) On calcule la couleur et on l'assigne au point de la grille considéré.

La Figure 2 page 4 présente un diagramme de classe simplifié.

On présente les diverses classes :

- **Vector3f** : c'est un vecteur 3D de float.
- **Ray3f** : un rayon avec une origine et une direction
- **Camera** : la caméra ou œil d'où on regarde la scène.

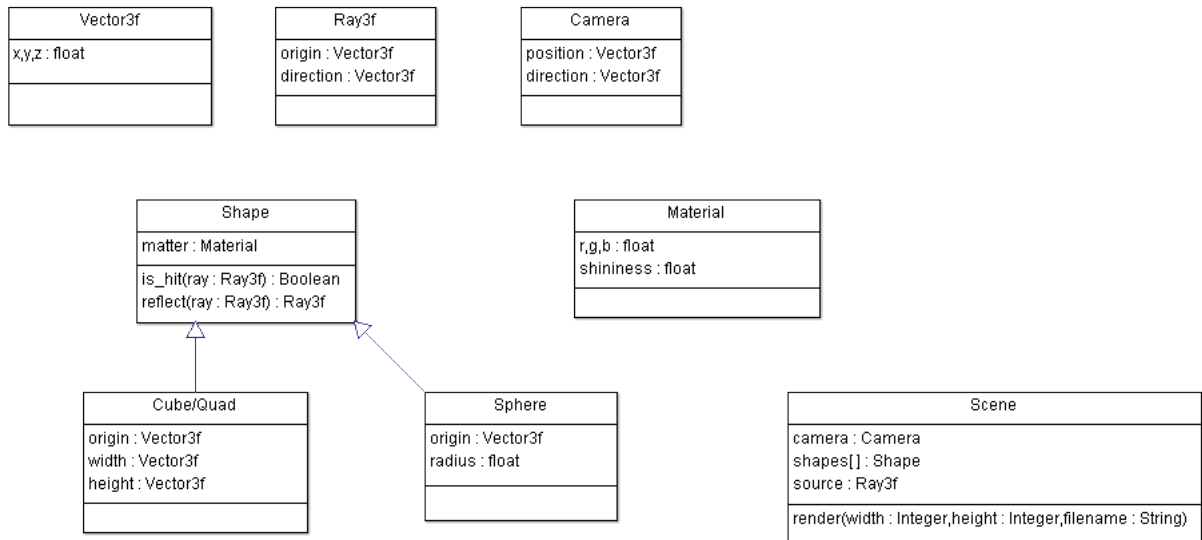


FIGURE 2 – Diagramme de classes

- **Material** : la couleur (uniforme) et un coefficient de luminosité entre 0 et 1 (0 signifie pas de réflexion (objet mat) et 1 signifie que le rayon est entièrement réfléchi (un miroir)).
- **Shape** : classe abstraite. La méthode `is_hit` teste si le rayon intersecte l'objet et la méthode `reflect` renvoie le rayon réfléchi.
- **Cube/Quad** : un cube ou un rectangle, défini par une origine (le centre) et la taille.
- **Sphere** : une sphère définie par une origine et un rayon.
- **Scene** : la scène qui comprend la caméra et les objets et la source de lumière. La méthode `render` définit la taille de la grille (donc de l'image) ainsi que le nom du fichier dans lequel on sauve l'image.

3 Sujet

La scène sera composée de 5 quadrilatères définissant une boîte, la caméra se trouvant au niveau du côté manquant (voir Figure 3).

On placera la source de lumière au niveau du quadrilatère supérieur, ainsi qu'une sphère et un cube.

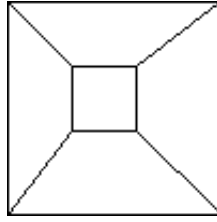


FIGURE 3 – Scene vide

Ecrire un programme en C++ correctement structuré, sans l'aide d'une quelconque bibliothèque extérieure au C++ qui affichera dans une fenêtre la scène calculée.

Pour afficher dans une fenêtre les caractères du travail ci-dessus (une fenêtre pour chacune de ces polices), on utilisera la bibliothèque SDL, dont l'utilisation sera faite grâce à une classe `Sdl` facilitant son usage.